



Desenvolvimento de um RISC-V RV32I Single Cycle

INSTRUTORES Elivander Judas Tadeu **Pereira**

Felipe Gustavo de Freitas **Rocha**

Letícia **Carneiro** de Souza

RESIDENTE Bárbara Mariana **Rocha** Raimundo

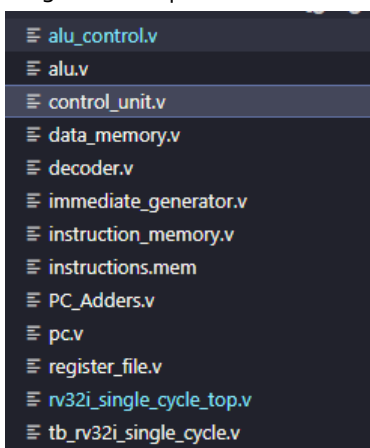
Especificações

A proposta do trabalho é desenvolver e simular um processador RISC-V de 32 bits baseado na ISA RV32I, utilizando uma arquitetura single-cycle e, assim, consolidar os conceitos aprendidos em sala de aula. A linguagem utilizada para o desenvolvimento do projeto foi Verilog, devido à maior familiaridade com ela.

Foi usado como base os códigos estudados em aula sobre a arquitetura MIPS, o livro Computer Organization and Design: The Hardware/Software Interface RISC-V Edition e os conceitos apresentados durante a disciplina. Com isso, foi possível desenvolver o projeto de acordo com o que foi solicitado.

Seguindo a organização sugerida no roteiro do trabalho, foram criados arquivos .v para os módulos mostrados na Figura 1, procurando implementar em cada um deles os requisitos descritos na seção "O que deve ser implementado" do roteiro.

Figura 1: Arquivos .v



Uma diferença em relação à organização apresentada no roteiro foi a criação do módulo PC_Adders, responsável pelo cálculo do próximo valor do PC em casos de desvio, enquanto o módulo PC funciona como um flip-flop para armazenar o próximo endereço da instrução.

Os testes para validação do projeto foram individualizados. A cada aluno foi atribuído um número e, a partir dele, foi possível obter os valores de A, B, C e MEM_ADDR. Esses parâmetros foram aplicados em um código desenvolvido em Assembly, do qual foram obtidas as instruções utilizadas na simulação.

Os valores atribuídos a mim foram: A = 9, B = 7, C = 20 e MEM_ADDR = 20.

1. Conversão das instruções para binário

A partir do código em Assembly e utilizando o cartão de referência do RISC-V, foi possível converter cada linha do programa em uma instrução binária de 32 bits. A título de exemplificação, será demonstrada abaixo a conversão da segunda instrução: *addi x2, x0, 7*

Figura 2: Organização dos 32 bits para cada tipo de instrução.

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12:10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Como *addi* é uma instrução do tipo I, o valor imediato (7) deve ser representado em 12 bits e ocupa os bits de 31 a 20 da instrução. Em seguida, são colocados os 5 bits referentes ao registrador fonte rs1. Nesse caso foi utilizado o registrador x0, cujo valor é sempre zero.

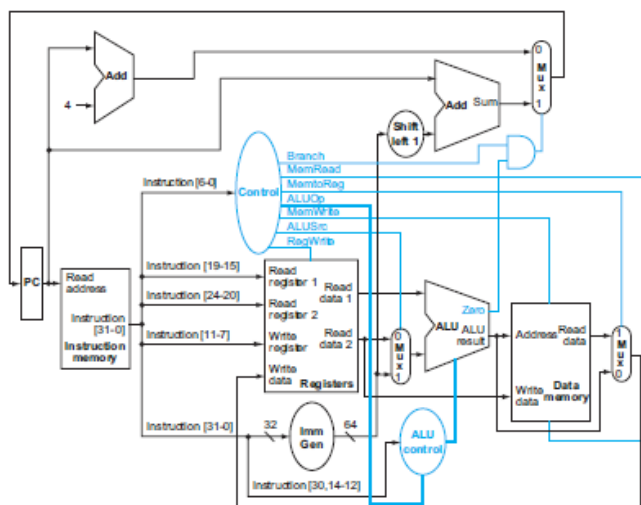
Após isso, são adicionados os 3 bits do campo funct3, responsáveis por especificar a operação executada. Para a instrução *addi*, o valor utilizado é 000. Na sequência, são inseridos os 5 bits correspondentes ao registrador de destino rd, que neste caso é o registrador x2.

Por fim, são adicionados os 7 bits do opcode, responsáveis por identificar o tipo da instrução. Para instruções aritméticas imediatas do tipo I, como *addi*, o opcode utilizado é 0010011. Ficando : 000000000111 00000 000 00010 0010011.

2. Breve explicação dos módulos

Uma base muito utilizada para a elaboração dos código foi o datapath do RISC-V presente no livro citado anteriormente e que pode ser observado abaixo.

Figura 3: Datapath



2.1. PC

Neste módulo, as entradas são o sinal de clock, responsável por controlar a atualização das instruções a cada ciclo de clock, conforme esperado para o caso da arquitetura single-cycle, o sinal de reset, utilizado para zerar o valor do PC (armazenar o endereço atual da instrução) , e o sinal nextPc, que contém o endereço da próxima instrução a ser executada.

2.2. PC_Adders

Inicialmente, a ideia para este módulo era implementar os dois somadores responsáveis pelo cálculo de $PC + 4$ e do endereço de desvio. Entretanto, optou-se por realizar apenas o cálculo do endereço de desvio. Dessa forma, o cálculo sequencial ($PC + 4$) passou a ser realizado diretamente no módulo topo, utilizando um mux para selecionar entre o próximo endereço sequencial e o endereço de desvio. Neste módulo é realizada apenas a soma entre o valor atual do PC e o imediato gerado pelo módulo ImmGen.

2.3. Decoder

O objetivo deste bloco é separar os 32 bits da instrução em seus respectivos campos, armazenando-os em sinais individuais. Dessa forma, são obtidos os campos opcode, funct3, funct7, rd, rs1 e rs2, que foi demonstrado na tabela da figura 2.

2.4. Instruction Memory

Neste bloco, é implementada uma memória de instruções de 32 bits com 256 posições, em que as instruções do programa são armazenadas. Como cada instrução possui 32 bits (4 bytes), o endereço é dividido por 4 para acessar corretamente a posição da memória.

2.5. Control Unit

No bloco da unidade de controle, o valor do opcode é utilizado para definir os sinais de controle. A partir dele, sinais como `alu_src`, `branch`, `jump`, `mem_read`, `mem_write` e `reg_write` são ativados ou desativados de acordo com o tipo da instrução executada. Por exemplo, quando `alu_src` = 1, o segundo operando da ALU passa a ser o imediato em vez do valor de um registrador.

2.6. Register File

Neste bloco é implementado o banco de registradores do processador, composto por 32 registradores de 32 bits. Os registradores `rs1` e `rs2` são utilizados para leitura, enquanto `rd` para escrita. Quando `reg_write` está ativo, o valor de `data_w` é armazenado no registrador indicado por `rd`. Além disso, o registrador `x0` é mantido sempre com valor zero, conforme definido na arquitetura RISC-V.

2.7. Register File

O bloco ALU Control é responsável por definir qual operação será executada pela ALU. A partir do sinal `alu_op`, gerado pela unidade de controle, e dos campos `funct3` e `funct7` da instrução, foi atribuído um sinal de controle de 4 bits utilizado pela ALU para selecionar as operações. Os valores utilizados para `alu_op` não seguem um padrão definido pela arquitetura RISC-V, portanto foi usado os valores apresentados na tabela da figura 4, retirada do livro anteriormente citado.

Figura 4: Alu op e alu control

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

2.7. ALU

É neste módulo em que as operações aritméticas e lógicas são realizadas, a partir do alucontrol. Os códigos de operação não presentes na tabela utilizada foram definidos arbitrariamente. Além disso, ele controla a flag zero, sinal de controle utilizado nas instruções de desvio.

2.8. Data Memory

Aqui foi implementada a memória de dados de 256 posições de 32 bits. A escrita é feita se o sinal `mem_write` estiver ativo, já a leitura é realizada quando `mem_read` está ativo. O endereço é dividido por 4 para acessar corretamente as instruções de 32 bits.

2.9. Immediate Generator

Ele tem a função de extrair os bits dos imediatos das instruções e expandi-los para 32 bits. Para os tipos I, S, B e J, é utilizada extensão de sinal, replicando o bit mais significativo do imediato nas posições à esquerda. Para exemplificar a reconstrução, no tipo S o imediato é dividido em duas partes: `inst[31:25]` (7 bits) e `inst[11:7]` (5 bits), totalizando 12 bits. Em seguida, são adicionados 20 bits de extensão de sinal, resultando em: `imm = {20{inst[31]}}, inst[31:25], inst[11:7]`.

2.10. Módulo topo

Esse módulo realiza a instanciação e interligação de todos os blocos citados e implementa a lógica de controle dos mux responsáveis pela seleção de caminhos de dados, como próximo PC, escrita no banco de registradores e escolha da fonte da ALU. Para tanto, foi utilizado como base o datapath da figura 3.

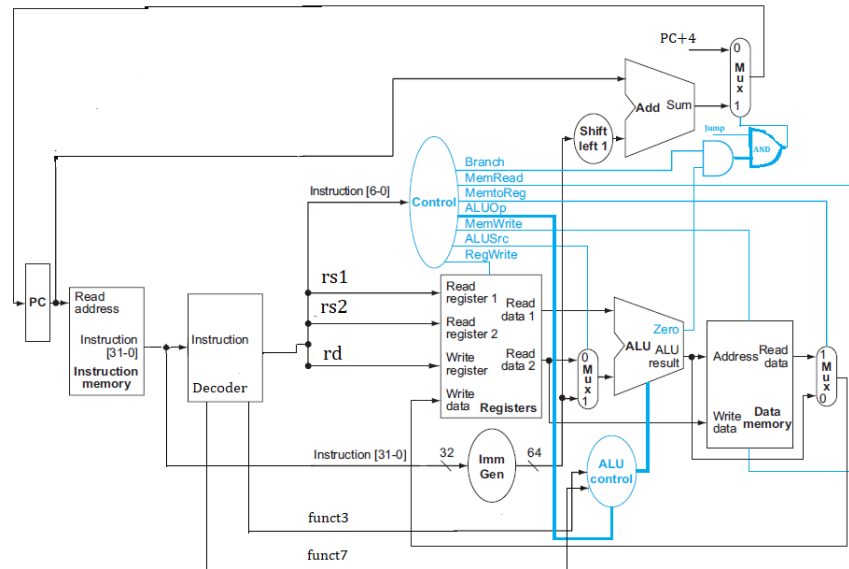
2.10. Modulo testbench

No testbench, é necessário apenas gerar o clock e o reset para realizar a validação.

3. Resultados

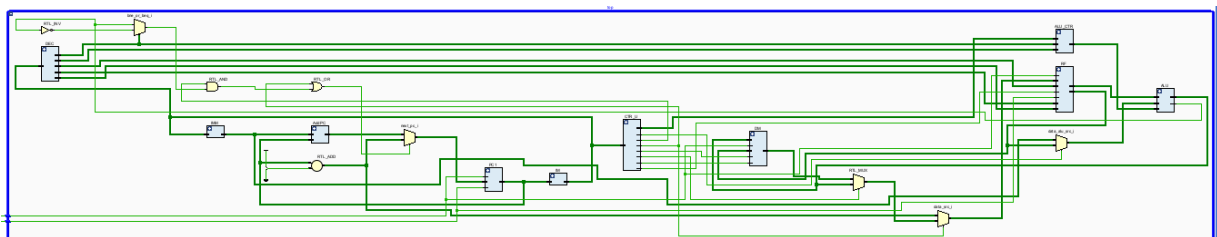
Após a elaboração dos módulos, foi desenvolvido um datapath simplificado a partir do diagrama do livro, o qual foi adaptado e modificado no Paint para representar o caminho de dados do projeto.

Figura 5: Caminho de dados



Além disso, para confirmar que o caminho de dados apresentado anteriormente está condizente com o projeto desenvolvido, foi gerado também o esquemático RTL por meio do Vivado. A partir dele, foi possível verificar que o datapath implementado corresponde ao diagrama apresentado previamente. A Figura 5 mostra o esquemático gerado. Embora o datapath seja extenso, a ferramenta permitiu, com o uso de zoom e navegação pelo circuito, confirmar que as conexões e módulos implementados estão coerentes com a arquitetura proposta.

Figura 6: Caminho de dados



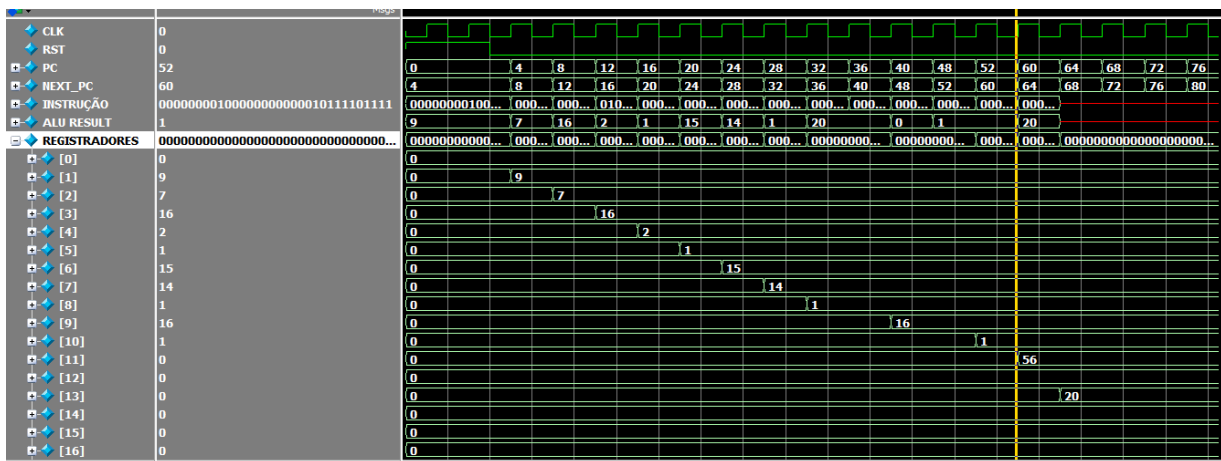
Para validar o funcionamento do RISC-V implementado, foram calculados previamente os valores esperados para cada instrução, para os sinais PC, para o banco de registradores e a saída da ALU. Esses valores calculados serviram como referência para validar o funcionamento. A tabela da figura 7 abaixo contém os valores esperados.

Figura 7: Tabela com valores esperados



Instrução Assembly	Instrução 32 bits (Binário)	PC	Next PC	Registrador	Valor Esperado Resultado ALU	Valor no Registrador (decimal)
addi x1, x0, 9	00000000100100000000000010010011	0	4	x1 (reg[1])	9	9
addi x2, x0, 7	000000001111000000000000100010011	4	8	x2 (reg[2])	7	7
add x3, x1, x2	00000000001000001000000110110011	8	12	x3 (reg[3])	(9+7) = 16	16
sub x4, x1, x2	01000000001000001000001000110011	12	16	x4 (reg[4])	(9-7) = 2	2
and x5, x1, x2	00000000001000001111001010110011	16	20	x5 (reg[5])	(1001 & 111) = 0001b = 1	1
or x6, x1, x2	00000000001000001110001100110011	20	24	x6 (reg[6])	(1001 111) = 1111b = 15	15
xor x7, x1, x2	00000000001000001100001110110011	24	28	x7 (reg[7])	(1001 ^ 111) = 1110 = 14	14
slt x8, x2, x1	0000000000100010010010000110011	28	32	x8 (reg[8])	(1001 < 111) = true = 1	1
sw x3, 20(x0)	00000000001100000010101000100011	32	36	-	20 (endereço)	Escrito na memória
lw x9, 20(x0)	0000000101000000001001001000011	36	40	x9 (reg[9])	20 (endereço)	16 (Dado da Memória)
beq x9, x3, label_ok	00000000001101001000010001100011	40	Pulou 1 (PC+4) = 48	-	16 - 16 = 0 - Desvio	-
addi x10, x0, 0	000000000000000000000010100010011	44	48	x10 (reg[10])	0 (Pulada)	0
addi x10, x0, 1	000000000000000000000010100010011	48	52	x10 (reg[10])	0+1 = 1	1
jal x11, fim	000000001000000000000001011101111	52	Pulou 1 (PC+8) = 60	x11 (reg[11])	next_pc + 4 = 52+56	56
addi x12, x0, 99	00000110001100000000011000010011	56	60	x12 (reg[12])	0 (Pulada)	0
addi x13, x0, 20	000000010100000000000011010010011	60	-	x13 (reg[13])	20	20

Figura 8: Waves com o valores obtidos



Pela figura 8, com as formas de onda obtidas na simulação, observa-se que os valores calculados apresentados na tabela estão de acordo com os resultados esperados. Além disso, é possível verificar que cada instrução é executada em um único ciclo de clock, conforme esperado para a arquitetura single cycle.

Adicionalmente, foi realizada a verificação dos dados armazenados na Data Memory e a análise do comportamento dos sinais de controle gerados pela Control Unit. Foi elaborada uma tabela com valores esperados, assim como o que foi feito anteriormente.

Figura 9: Tabela com valores esperados



Atividade Conceitual / Prática

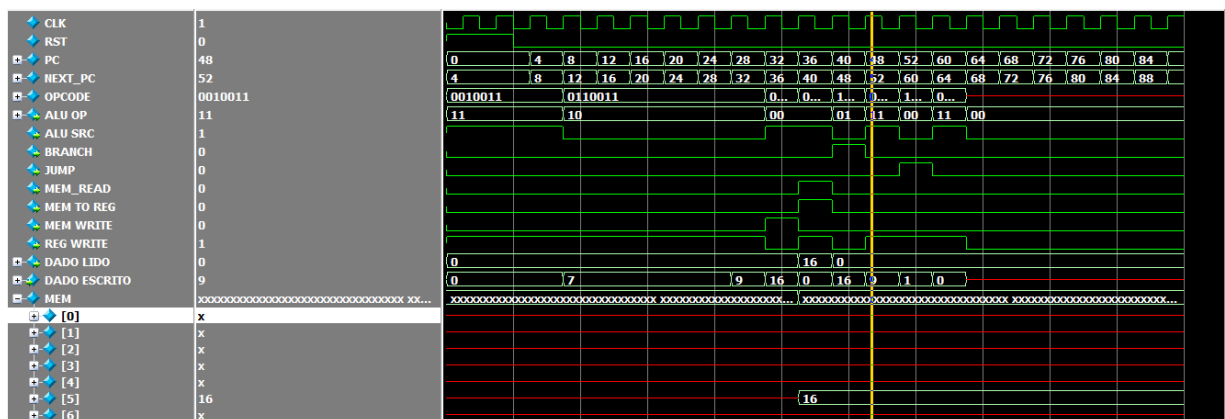
Instrução Assembly	PC	Next PC	Memória	OPCode	ALU OP	ALU SRC	BRANCH	JUMP	MEM_READ	MEM To Reg	MEM WRITE	RegWrite	Dado Lido	Dado Escrito
addi x1, x0, 9	0	4		10011	2'b11	1	0	0	0	0	0	1	-	-
addi x2, x0, 7	4	8		10011	2'b11	1	0	0	0	0	0	1	-	-
add x3, x1, x2	8	12		110011	2'b10	0	0	0	0	0	0	1	-	-
sub x4, x1, x2	12	16		110011	2'b10	0	0	0	0	0	0	1	-	-
and x5, x1, x2	16	20		110011	2'b10	0	0	0	0	0	0	1	-	-
or x6, x1, x2	20	24		110011	2'b10	0	0	0	0	0	0	1	-	-
xor x7, x1, x2	24	28		110011	2'b10	0	0	0	0	0	0	1	-	-
sllt x8, x2, x1	28	32		110011	2'b10	0	0	0	0	0	0	1	-	-
sw x3, 20(x0)	32	36		100011	2'b00	1	0	0	0	0	1	0	-	16
lw x9, 20(x0)	36	40	mem[5] = 16	000011	2'b00	1	0	0	1	1	0	1	16	-
beq x9, x3, label_ok	40	Pulou 1 (PC+4) = 48		1100011	2'b01	0	1	0	0	0	0	0	-	-
addi x10, x0, 0	44	48		0010011	2'b11	1	0	0	0	0	0	1	-	-
addi x10, x0, 1	48	52		0010011	2'b11	1	0	0	0	0	0	1	-	-
jal x11, fim	52	Pulou 1 (PC+8) = 60		1101111	2'b00	0	0	1	0	0	0	1	-	-
addi x12, x0, 99	56	60		0010011	2'b11	1	0	0	0	0	0	1	-	-
addi x13, x0, 20	60	-		0010011	2'b11	1	0	0	0	0	0	1	-	-

Na tabela acima tem o comportamento dos sinais de controle gerados pela unidade de controle para cada instrução do programa de teste. O sinal RegWrite é ativado sempre que a instrução precisa armazenar dados de um resultado no banco de registradores. Já o sinal ALUSrc vale 1 quando a ALU utiliza um imediato e 0 quando utiliza um dado vindo do banco de registradores. Já MemRead e MemWrite são ativados em operações de leitura e escrita na memória, respectivamente. O sinal MemToReg é utilizado quando o dado vindo da memória deve ser escrito em um registrador. Os sinais Branch e Jump são ativados em instruções de desvio.

Os sinais com mais de um bit, como ALUOp, seguem a codificação apresentada anteriormente. O campo Opcode segue a codificação padrão do RISC-V.

Por fim, na instrução sw x3, 20(x0), o valor 16 é armazenado no endereço 20. Como a memória é endereçada por palavras de 32 bits, o índice interno é obtido dividindo o endereço por 4, resultando em mem[5].

Figura 10: Waves com o valores obtidos





Pela figura 10, com as formas de onda obtidas na simulação, observa-se que os valores calculados apresentados na tabela estão de acordo com os resultados esperados.

4. Conclusão

Com as duas simulações apresentadas nos resultados, foi possível validar o funcionamento do projeto, já que os demais blocos, mesmo não sendo detalhados individualmente nas formas de onda, influenciam diretamente no resultado final da execução das instruções. Assim, caso algum módulo apresentasse erro, isso seria refletido em uma das simulações realizadas.

Por fim, a realização deste trabalho foi importante para consolidar os conhecimentos sobre a arquitetura RISC-V, compreender o fluxo de execução das instruções, praticar Assembly e exercitar o desenvolvimento em Verilog.

Referência

- [1]. Computer Organization and Design: The Hardware/Software Interface RISC-V Edition.
- [2]. https://github.com/Hola39e/MIPS_Multi_Implementation/tree/master.
- [3]. https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf
- [4]. <https://www.eq.bucknell.edu/~csci206/riscv-converter/>